

Keeping Up with the Times – How Universities Can Integrate LLMs into Software Engineering Education to Better Prepare Students for an AI-Driven Future

Section 1: Research Methods Summary

The driving research question of this paper was, **“How can universities integrate generative AI (such as ChatGPT, Gemini, Claude) into their computer science curricula by teaching students proper applications and potential drawbacks of these models for software development, rather than strictly banning them?”**

The motivation for this inquiry comes from the rapid increase in the use of Large Language Models (LLMs) to generate code for students and industry professionals alike, and their specific usefulness in the field of software engineering. Consumer-facing implementations of LLMs started seeing widespread usage around late 2022 during my freshman year, and since then many students use them everyday to complete coursework. This of course was met by a rapid response by UMass Amherst to strictly ban their usage, being considered under the Academic Honesty policies as cheating. However, some professors in computer science and other departments at UMass have recently started to allow the use of AI with proper citations.

The software engineering field has rapidly moved toward integrating AI workflows into the development cycle, which begs the question of how university computer science and software engineering programs can alter their pedagogical approaches to set their students up for professional success in the future.

The sources used in this paper seek to answer various related and subsidiary questions of how generative AI has and will continue to alter the landscape of software engineering education. They were chosen among results from Google Scholar and the ACM Digital Library using the search query, **“(“AI” OR “Artificial Intelligence” OR “Large Language Models”) AND (“Software Development” OR “Software Engineering”) AND (“University” AND “Curriculum”).** The papers consulted represent a diverse range of methods to explore this topic, including literature reviews [1], empirical studies [4], and educational workshops [5]. They also span the entirety of the recent era defined by the popularity of LLMs, the earliest including faculty interviews from early 2023 (shortly after the release of ChatGPT) [4], and the most recent from February 2025 being the beginning of a three year curriculum reform project at the University of Missouri [2].

2: Objective Findings

Goals of Software Engineering Education

To understand how to adapt the current framework of software engineering education, we first need an understanding of its underlying goals. Kirova et. al. [3] define a four-part approach to this: convey to the student the foundations of software engineering, develop and strengthen their

skills in the core aspects of software engineering, prepare students for a career in software development, and place software engineering within the context of the university's program as well as within a broader scope of computer science and their education. They elaborate, stating that students should be introduced to the concepts and motivation of the software life cycle, as well as common development frameworks such as AGILE and Scrum, additionally including ideas such as test-driven development, software architecture, requirements analysis, specification, and documentation generation. To teach and strengthen relevant career skills, the program should involve a large-scale team project involving actual client interactions, multiple iterations, a minimum viable product (MVP), and deployment. To build professional competence, the program should place a focus on developing critical thinking and proficiency with soft skills including interpersonal communication. Additionally, the program should illustrate the connection of these topics to other computer science courses taken by the student and how software engineering fits within a broader technical and social context.

Any future amendments to software engineering curricula involving LLMs may aim to emulate these four major goals, or perhaps suggest a different vision for computer science education. Chen and Alias [1] suggest that, "the cultivation goals of software engineering and related programs need appropriate adjustments to better adapt to the current technological environment and market demands." They seek to build upon the traditional goals by adding that more focus should be placed on enhancing students' innovation capabilities in the field of AI and its applications. The curriculum should also facilitate interdisciplinary skills, especially with data science and machine learning. Additionally, it should teach professional ethics and responsibility including compliance with laws and regulations. Lastly, it should expand student's global perspectives and teamwork abilities. They suggest that these updated goals will be better suited for a world with LLMs and better prepare students for career opportunities.

Practical Applications of LLMs in Software Engineering

To properly educate students on the use of LLMs, we must first understand their current practical applications within the field of software engineering. So far, generative AI seems to be useful in almost every step of the software development life cycle, from project idea generation to specification and functional requirements, code generation, automated test generation, refactoring, styling consistency, database seeding, and advice with client presentations [1,3]. In fact, Kozov et. al. [5] devised a new model of the software life cycle that could be taught to students that includes AI integration for various steps.

Generative AI models seem to have fundamentally changed how developers write software. Auto-completion tools such as GitHub Copilot increase programmer efficiency and code quality by automating repetitive work and reducing errors [1,3]. With AI able to do the heavy-lifting and reducing the cognitive load of developers, more time can be spent on higher-level design. Cross-platform and multi-language development are made easier when LLMs can easily migrate code between languages. LLMs have also been applied to areas of project management including evaluating task complexity and enhancing team cooperation and performance. They can also be

used for personalized learning in either a professional setting or a classroom. Tools such as ChatGPT can be used to generate examples and answer specific questions a developer might have about a new framework or language they're learning, or to clarify the lessons taught in one of their courses.

Dangers and Limitations of using LLMs in Software Engineering

To properly educate students on the use of LLMs, we must also understand the dangers and limitations of applying them to software development. One issue is the common occurrence of LLMs "hallucinating." As stated by Kozov et. al., "It is important to note that all LLMs often hallucinate, and because of that they are not reliable for checking facts and truth" [5]. Because of the non-deterministic nature of LLMs, responses may differ when given the same prompt, contain erroneous information and citations, or exhibit biases from the training data. These issues can also arise from the model conflating meanings, lacking an understanding of the greater context of a task, or failing to consider serious security risks [1,3,4]. In the context of code generation, this can lead to inaccurate code, security vulnerabilities, and inconsistent styling, especially when dealing with newer libraries or frameworks that may not be represented in the training data [4].

These limitations have serious ethical and legal implications. If an LLM were to "hallucinate" or introduce a security flaw in production code, millions of people could be affected. One area of concern is the collection of data for model training [1,3]. Some LLMs may have been trained on data which was scraped recklessly without regard for privacy or copyright. In the case of privacy, personal information may leak in the model's responses. In the case of copyright, the model may regurgitate copyrighted code and potentially violate software licenses. Both possibilities present a difficult choice of who would be held at legal fault in a potential lawsuit emerging from AI-generated code or content. LLMs can also be used intentionally for unethical purposes such as generating misleading info or to develop malware, again bringing into question whether the blame is on AI companies or end users.

Another big worry, especially in the case of novice software developers, is developing an over-reliance on AI tools [1]. Kozov et. al. states, "A big detriment of using AI by first and second year students is their lack of understanding in the subjects where they are trying to use AI. This sadly leads to less problem-solving thinking and doing actual effort to work on tasks. This in turn leads to less brain development and lack of practical skills" [5]. If beginner programmers use these tools for problem solving and code generation, they may fail to develop a deep understanding of computing fundamentals and necessary coding skills. They may also become less independent and less adaptive, being unable to solve coding problems without an AI assistant. This can also impact algorithm design skills, having to rely on ready-made outputs from LLMs, making it more difficult to take on novel or complex problems. Over reliance on LLMs may also affect the coding style of a budding developer if they start to mimic the style of AI-generated code, which can often favor complex solutions or be inconsistent with syntax across prompts.

Methods of Integrating LLMs into University Software Engineering Programs

Having considered the goals of software engineering education, the practical applications of LLMs in software development, and their associated limitations, we can now consider changes to university curricula that integrate all of these factors. A useful approach might be to look at programs where this has already happened. Hare et. al. detail their NSF-sponsored department-level renovation at the University of Missouri Kansas City, which began in early 2025 and will last three years [2]. Their core approach consists of three parts, “(a) enhancing students' problem-solving and programming skills by leveraging LLMs as a learning tool in core programming courses, (b) improving students' software development skills by integrating LLM-powered tools into the software engineering course sequence, and (c) educating students on ethical and responsible AI practices.” They plan to implement these changes in a variety of undergraduate courses ranging from introductory computer science to a software engineering capstone project.

A similar approach has been suggested for adjusting existing courses to include LLMs [1]. Each core computer science course such as Data Structures or Algorithms should be updated to include an LLM unit that would detail how AI tools can be applied to that subject. Additionally, it's been suggested that the foundational learning outcomes of the classes will be altered as well [1,3]. For example, much like how modern calculators made conceptual understanding of math far more important than manual arithmetic skills, LLMs may shift the focus of coding education from studying detail-oriented implementation and syntax to high-level design and broader concepts. This would also mean less time learning code synthesis, and much more time spent on code review, testing, validation, verification, and quality assurance to minimize risks created by AI. This puts an importance on not just technical skills, but critical thinking skills and the ability to properly supervise AI-driven projects.

Several new courses have also been proposed for software engineering programs [1]. A “Large Language Model Applications” course would focus on “basic theories, algorithm principles, and applications of LLMs in various fields.” A “Professional Practice” course would teach the current state of AI usage in the industry and focus on practical applications of LLMs to software engineering projects. And a required “Artificial Intelligence Ethics and Law” class would discuss pertinent ethical and legal issues regarding AI usage both for personal and professional use.

These various ideas for integrating LLMs into university programs would not just affect the content of software engineering education, but its pedagogy as well [3]. Assignments will teach the students not just how to use AI, but will actually require their usage to complete. For example, a student may be asked to identify potential vulnerabilities in AI-generated code, or write their own solution and compare it to that of an LLM. If professors are looking for original student work, they will need to implement extra measures to mitigate the use of AI tools. This could involve asking open-ended questions that require a specific context from the class that AI may struggle to answer, using bespoke libraries for coding assignments so that the AI has a lack of familiarity with it, or focusing more on oral, video, and image submissions from

process-based work. Exams will likely either have to take place in person on paper, or in a locked-down computer lab environment to mitigate any use of AI [1]. It's also been suggested that fundamental computing techniques are practiced and mastered in beginner courses before allowing the use of AI. In intermediate or advanced classes, a flipped classroom model could be used where students study the material on their own, then work on supervised AI-driven projects during class time. This would be made more effective if students were taught to guide their independent learning by using LLMs as a personalized instructor.

3: Subjective Conclusion

These findings exist within the larger context of proceedings on computer science education at the intersection of pedagogy techniques and industry preparation for students. The interviews conducted by Lau and Guo represent a unique snapshot of how LLMs were first viewed in academia during early 2023 [4]. It's interesting to see how these initially mixed opinions on their utility have evolved into a general consensus both in academia and the industry that AI is here to stay. Their predictions aged well, "it is inevitable that AI tools will become more widespread, so it is futile to resist." We've since gone from sudden bans of AI and mixed scepticism and intrigue to the first programs that will permanently enshrine LLMs in the halls of academia [2].

Furthermore, the LLM-driven changes in computer science education suggest the presence of a larger paradigm shift in the software engineering world. The role of a developer is shifting from that of a technical engineer focused on details and semantics to that of a product manager focused on communication, design, and critical thinking. The relevant skill set then pivots from code writing to code understanding and testing.

An additional area of inquiry also arose from the research about how LLMs can aid instructors in teaching computer science classes [1,4]. Some instructors brainstormed ways to use AI to enhance their teaching methods such as automated grading and feedback for student assignments, generating variants of problem sets, acting as a personal tutor for students, or class management including custom learning plans for each student and data analysis of student performance. These concepts could be explored further in future research.

Considering the sources discussed, I propose the following guidelines to amend university software engineering programs (or general computer science programs which have a software engineering component) for the age of LLMs. Firstly, the program should focus on teaching students the *theory* of AI usage in development. Learning outcomes may seek to have students understand "Why use AI in the first place?", "What ethical or legal issues could AI usage create for a company?", or "What risks are incurred when using AI to develop software?" The program should also focus on teaching students the *practice* of AI usage in development. Learning outcomes will seek to teach students the technical skills of developing software with AI through project-based learning and how to automate low-level tasks. This should include prompt engineering and a large-scale team-based software project.

For both approaches, these learning outcomes should be distributed throughout the course roadmap of the student and integrate within each discipline they encounter. A new low-credit required class on LLM usage may also be added to the course roadmap to give sufficient time to explore these ideas, especially in their own context. Students should be encouraged to do their own research into LLM usage and consider the pros and cons when deciding whether or not to use AI tools for a project or assignment.

At the end of such a program a student should be (a) proficient at using LLMs for development, (b) proficient at developing in the absence of LLMs, (c) able to determine when AI usage is appropriate for a given project, and (d) able to identify flaws in AI-generated outputs.

There's also a few important things to note. With a range of assignments from requiring AI usage to banning it, the guidelines will need to be explicitly clear for every assignment to prevent confusion or accidental cheating by students. Similarly, assignments that utilize or allow the usage of AI will need different grading standards than those who don't. These policies should always be clearly stated, and rubrics that account for AI will have to require proper citations of generated code or text, and the relevant prompts. Three types of assignments become apparent, those who require the usage of AI, those who allow it if the student chooses, and those who do not allow it. The second type, because of its unsupervised nature, should be put off until after the first year or two of the program, so the student has a chance to develop strong programming fundamentals, and to avoid overreliance on AI tools.

A positive side effect of program adaptations for AI is that they will likely lead to improved course pedagogy by forcing professors to reevaluate their teaching methods and restructure any outdated practices for a modern setting. The promotion of soft skills such as critical thinking, design, and independent inquiry is also a huge plus, and often an underdeveloped skill in computing professionals. Ideally, a curriculum that follows these guidelines will best prepare software engineering students to practice independent thinking, good judgement, and technical proficiency in a rapidly changing industry.

References

- [1] Ping Chen and Syazwina Binti Alias. 2024. Opportunities and challenges in the cultivation of software development professionals in the context of large language models. In *ISAIE '24: Proceedings of the 2024 International Symposium on Artificial Intelligence for Education*, Association for Computing Machinery, 259–267. DOI:<https://doi.org/10.1145/3700297.3700342>
- [2] Brian K Hare, Joan Gladbach, Shah S Jawad, and Dianxiang Xu. 2025. Building AI-Powered responsible workforce by integrating large language models into computer science curriculum. In *SIGCSE 2024: Proceedings of the 55th ACM Technical Symposium on Computer Science Education*, Association for Computing Machinery, Pittsburgh, PA, USA, 1709–1710. DOI:<https://doi.org/10.1145/3641555.3704749>
- [3] Kirova, Vassilka D, Cyril S Ku, Joseph R Laracy, and Thomas J Marlowe. 2024. Software engineering education must adapt and evolve for an LLM environment. In *Proceedings of ACM Technical Symposium on Computer Science Education (SIGCSE '25)*, Association for Computing Machinery, Portland, OR, USA, 666–672. DOI:<https://doi.org/10.1145/3626252.3630927>
- [4] Sam Lau and Philip Guo. 2023. From “Ban It Till We Understand It” to “Resistance is Futile”: How university programming instructors plan to adapt as more students use AI code generation and explanation tools such as ChatGPT and GitHub copilot. In *ICER '23: Proceedings of the 2023 ACM Conference on International Computing Education Research*, Association for Computing Machinery, Chicago, IL, USA, 106–121. DOI:<https://doi.org/10.1145/3568813.3600138>
- [5] Vasil Kozov, Galina Ivanova, and Desislava Atanasova. 2024. Practical Application of AI and Large Language Models in Software Engineering Education. *International Journal of Advanced Computer Science and Applications* 15, 1 (January 2024). DOI:<https://doi.org/10.14569/ijacsa.2024.0150168>